

06 - Progettazione architeturale

Francesco Betti Sorbelli

Università degli studi di Perugia. `francesco.bettisorbelli@unipg.it`



1 Decisioni di progettazione architeturale

2 Viste architeturali

3 Modelli architeturali

- La progettazione architetturale si occupa di capire come deve essere organizzato un sistema SW e di progettare la struttura complessiva di tale sistema.
- È il collegamento critico tra la progettazione e l'ingegneria dei requisiti, in quanto identifica i principali componenti strutturali di un sistema e le relazioni tra di essi.
- Il risultato del processo di progettazione architetturale è un modello architetturale che descrive come il sistema è organizzato come un insieme di componenti comunicanti.

- È generalmente accettato che una fase iniziale dei processi agili consista nel progettare un'architettura generale dei sistemi.
- La rifattorizzazione (*refactoring*) dell'architettura del sistema è solitamente costosa perché riguarda molti componenti del sistema.

- L'architettura in piccolo si occupa dell'architettura dei singoli programmi. A questo livello, ci occupiamo del modo in cui un singolo programma viene scomposto in componenti.
- L'architettura nelle grandi aziende si occupa dell'architettura di sistemi aziendali complessi che includono altri sistemi, programmi e componenti di programmi. Questi sistemi aziendali sono distribuiti su diversi computer, che possono essere di proprietà e gestiti da diverse aziende.

- Comunicazione con gli stakeholder
 - L'architettura può essere utilizzata come argomento di discussione dagli stakeholder del sistema.
- Analisi del sistema
 - Significa che è possibile analizzare se il sistema può soddisfare i suoi requisiti non funzionali.
- Riutilizzo su larga scala
 - L'architettura può essere riutilizzata in una serie di sistemi.
 - Possono essere sviluppate architetture di linee di prodotto.

- I diagrammi a blocchi semplici e informali che mostrano entità e relazioni sono il metodo più utilizzato per documentare le architetture SW.
- Ma questi sono stati criticati perché mancano di semantica, non mostrano i tipi di relazioni tra le entità né le proprietà visibili delle entità nell'architettura.
- Dipende dall'uso dei modelli architetture. I requisiti per la semantica dei modelli dipendono dal modo in cui i modelli vengono utilizzati.
- Esempio: Diagrammi a scatola e a linee
 - Molto astratti: non mostrano la natura delle relazioni tra i componenti né le proprietà visibili all'esterno dei sottosistemi.
 - Tuttavia, è utile per la comunicazione con le parti interessate e per la pianificazione del progetto.

- Come modo per facilitare la discussione sul progetto del sistema
 - Una vista architettuale di alto livello di un sistema è utile per la comunicazione con gli stakeholder del sistema e per la pianificazione del progetto, perché non troppo dettagliata.
 - Gli stakeholder possono relazionarsi con essa e comprendere una visione astratta del sistema.
 - Possono quindi discutere del sistema nel suo complesso senza essere confusi dai dettagli.
- Come modo di documentare un'architettura che è stata progettata
 - L'obiettivo è quello di produrre un modello di sistema completo che mostri i diversi componenti di un sistema, le loro interfacce e le loro connessioni.

Argomenti trattati

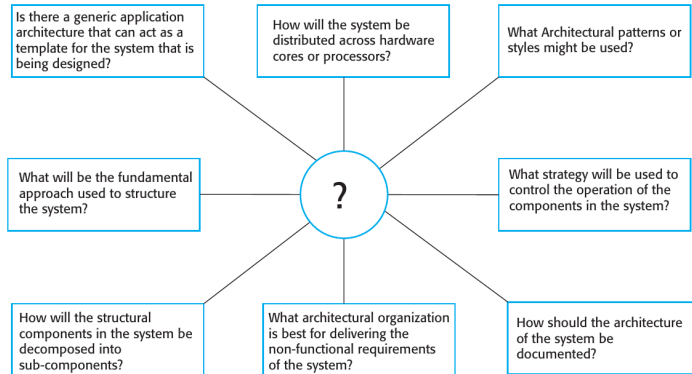
1 Decisioni di progettazione architetturale

2 Viste architetturali

3 Modelli architetturali

Decisioni di progettazione architetturale

- La progettazione architetturale è un processo creativo, quindi il processo varia a seconda del tipo di sistema da sviluppare.
- Tuttavia, alcune decisioni comuni a tutti i processi di progettazione influiscono sulle caratteristiche non funzionali del sistema.



Riutilizzo dell'architettura

- I sistemi dello stesso dominio hanno spesso architetture simili che riflettono i concetti del dominio.
- Le linee di prodotti applicativi sono costruite attorno a un'architettura di base con varianti che soddisfano particolari esigenze dei clienti.
- L'architettura di un sistema può essere progettata in base a uno o più modelli o "stili" architettureali.
 - Questi catturano l'essenza di un'architettura e possono essere istanziati in modi diversi.

Architettura e caratteristiche del sistema

- Prestazioni
 - Localizzare le operazioni critiche e ridurre al minimo le comunicazioni. Utilizzare componenti a grana grossa piuttosto che a grana fine.
- Sicurezza (Security)
 - Utilizzate un'architettura a strati con gli asset critici nei livelli più interni.
- Sicurezza (Safety)
 - Localizzare le funzioni critiche per la sicurezza in un numero ridotto di sottosistemi.
- Disponibilità
 - Includere componenti ridondanti e meccanismi di tolleranza ai guasti.
- Manutenibilità
 - Utilizzare componenti a grana fine, possibilmente sostituibili.

Argomenti trattati

1 Decisioni di progettazione architeturale

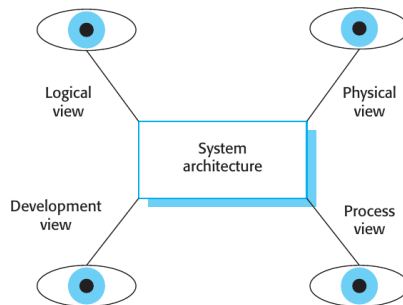
2 Viste architetture

3 Modelli architetture

Viste architetture

- Quali punti di vista o prospettive sono utili quando si progetta e si documenta l'architettura di un sistema?
- Quali notazioni si dovrebbero usare per descrivere i modelli architetture?
- Ogni modello architetture mostra solo una vista o una prospettiva del sistema.
 - Potrebbe mostrare come un sistema è scomposto in moduli, come interagiscono i processi di esecuzione o i diversi modi in cui i componenti del sistema sono distribuiti su una rete.
 - Sia per la progettazione che per la documentazione, di solito è necessario presentare più viste dell'architettura del SW.

Modello a 4 + 1 viste dell'architettura del SW



- Vista logica: mostra le astrazioni chiave del sistema come oggetti o classi di oggetti.
- Vista di processo: mostra come, a tempo di esecuzione, il sistema sia composto da processi interagenti.
- Vista di sviluppo: mostra come il SW viene scomposto per lo sviluppo.
- Vista fisica: mostra l'HW del sistema e come i componenti SW sono distribuiti tra i processori del sistema.
- Casi d'uso o scenari correlati (+1)

Argomenti trattati

1 Decisioni di progettazione architetturale

2 Viste architetturali

3 Modelli architetturali

Modelli architetturali

- I modelli sono un mezzo per rappresentare, condividere e riutilizzare la conoscenza.
- Un modello architetturale è una descrizione stilizzata di una buona pratica di progettazione, che è stata provata e testata in ambienti diversi.
- I modelli dovrebbero includere informazioni su quando sono utili e quando non lo sono.
- I modelli possono essere rappresentati utilizzando descrizioni tabellari e grafiche.

Il modello Model-View-Controller (MVC)

Nome	MVC (Modello-Vista-Controllore)
Descrizione	Separa la presentazione e l'interazione dai dati del sistema. Il sistema è strutturato in tre componenti logici che interagiscono tra loro. Il componente Model gestisce i dati del sistema e le operazioni associate su tali dati. Il componente View definisce e gestisce la presentazione dei dati all'utente. Il componente Controller gestisce l'interazione con l'utente (ad esempio, pressione di tasti, clic del mouse, ecc.) e passa queste interazioni alla View e al Model (vedi Figura 1).
Esempio	La Figura 2 mostra l'architettura di un sistema applicativo basato sul Web e organizzato secondo il modello MVC.
Quando viene utilizzato	Si usa quando esistono più modi per visualizzare e interagire con i dati. Si usa anche quando non si conoscono i requisiti futuri per l'interazione e la presentazione dei dati.
Vantaggi	Consente di modificare i dati indipendentemente dalla loro rappresentazione e viceversa. Supporta la presentazione degli stessi dati in modi diversi, con le modifiche apportate in una rappresentazione che vengono mostrate in tutte.
Svantaggi	Può comportare codice aggiuntivo e complessità del codice quando il modello dei dati e le interazioni sono semplici.

L'organizzazione del Model-View-Controller

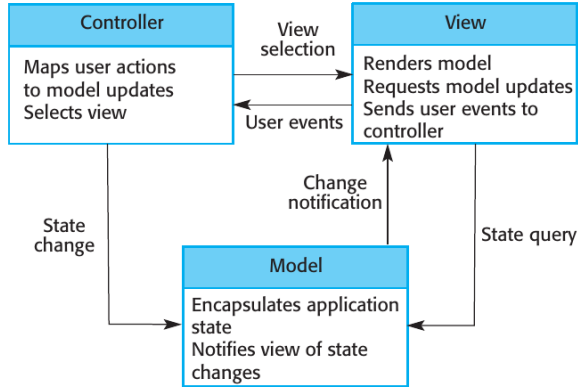


Figura 1

Architettura delle applicazioni web con il pattern MVC

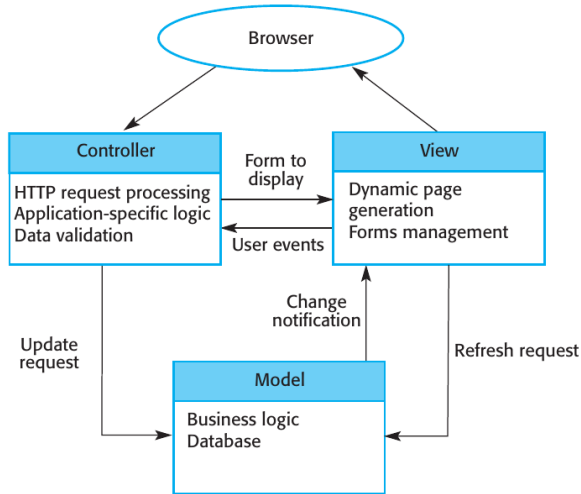


Figura 2

Architettura a strati

- Utilizzato per modellare l'interfacciamento dei sottosistemi.
- Organizza il sistema in un insieme di livelli (o macchine astratte), ciascuno dei quali fornisce un insieme di servizi.
- Supporta lo sviluppo incrementale di sottosistemi in diversi livelli. Quando l'interfaccia di un livello viene modificata, viene influenzato solo il livello **adiacente**.
- Tuttavia, spesso è artificiale strutturare i sistemi in questo modo.

Il modello di architettura a strati

Nome	Architettura a strati
Descrizione	Organizza il sistema in livelli con funzionalità correlate associate a ciascun livello. Un livello fornisce servizi al livello superiore, per cui i livelli più bassi rappresentano servizi fondamentali utilizzati in tutto il sistema (Figura 3).
Esempio	Un modello stratificato di un sistema per la condivisione di documenti protetti da copyright conservati in diverse biblioteche (Figura 4).
Quando viene utilizzato	Si usa quando si costruiscono nuove strutture su sistemi esistenti; quando lo sviluppo è distribuito tra diversi team e ogni team è responsabile di un livello di funzionalità; quando c'è un requisito di sicurezza a più livelli.
Vantaggi	Consente la sostituzione di interi strati, purché venga mantenuta l'interfaccia. È possibile fornire strutture ridondanti (ad esempio, l'autenticazione) in ogni strato per aumentare l'affidabilità del sistema.
Svantaggi	In pratica, fornire una separazione netta tra i livelli è spesso difficile e un livello di alto livello può dover interagire direttamente con i livelli inferiori piuttosto che attraverso il livello immediatamente inferiore. Le prestazioni possono essere un problema a causa dei molteplici livelli di interpretazione di una richiesta di servizio che viene elaborata a ogni livello.

Un'architettura generica a strati

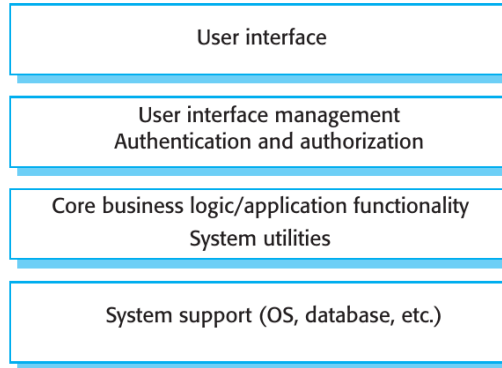


Figura 3

L'architettura del sistema iLearn

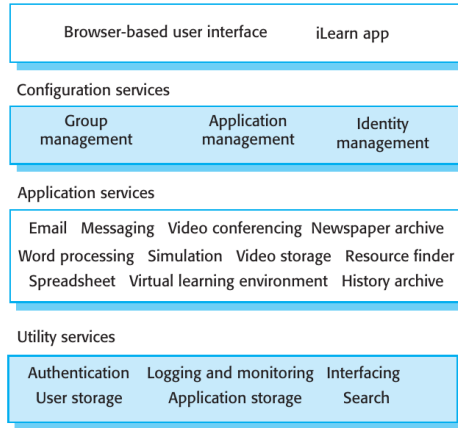


Figura 4

Architettura del repository

- I sottosistemi devono scambiarsi dati. Ciò può avvenire in due modi:
 - I dati condivisi sono conservati in un database o repository centrale e possono essere consultati da tutti i sottosistemi;
 - Ogni sottosistema mantiene il proprio database e passa i dati esplicitamente agli altri sottosistemi.
- Quando si devono condividere grandi quantità di dati, il modello di condivisione più utilizzato è quello del repository, un meccanismo efficiente di condivisione dei dati.

Lo schema del repository

Nome	Repository
Descrizione	Tutti i dati di un sistema sono gestiti in un archivio centrale accessibile a tutti. I componenti non interagiscono direttamente, ma solo attraverso esso.
Esempio	La Figura 5 è un esempio di IDE in cui i componenti utilizzano un archivio di informazioni sulla progettazione del sistema. Ogni strumento SW genera informazioni che possono essere utilizzate da altri strumenti.
Quando viene utilizzato	Quando si ha un sistema in cui vengono generati grandi volumi di informazioni che devono essere conservate per molto tempo. Può essere utilizzato anche in sistemi guidati dai dati, in cui l'inclusione dei dati nel repository attiva un'azione.
Vantaggi	I componenti possono essere indipendenti: non hanno bisogno di conoscere l'esistenza di altri componenti. Le modifiche apportate da un componente possono essere propagate a tutti i componenti. Tutti i dati possono essere gestiti in modo coerente, poiché si trovano tutti in un unico luogo.
Svantaggi	Il repository è un <i>single point of failure</i> , quindi i problemi nel repository si ripercuotono sull'intero sistema. Possono esserci inefficienze nell'organizzazione di tutte le comunicazioni attraverso il repository. La distribuzione del repository su diversi computer può essere difficile.

Un'architettura di repository per un IDE

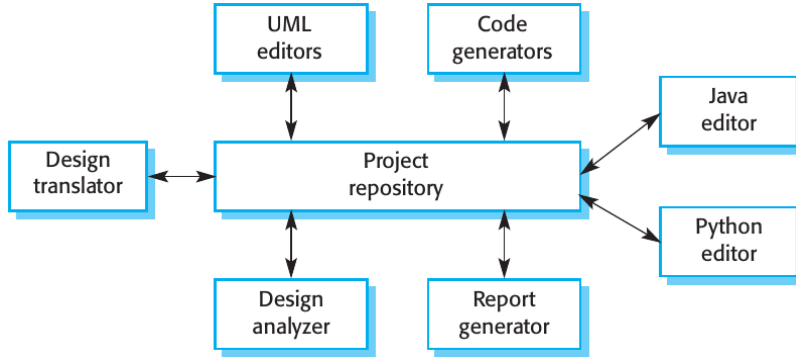


Figura 5

Architettura client-server

- Modello di sistema distribuito che mostra come i dati e l'elaborazione sono distribuiti su una serie di componenti.
 - Può essere implementato su un singolo computer.
- Insieme di server autonomi che forniscono servizi specifici come la stampa, la gestione dei dati, ecc.
- Insieme di client che si rivolgono a questi servizi.
- Rete che consente ai client di accedere ai server.

Il modello client-server

Nome	Cliente-server
Descrizione	In un'architettura client-server, le funzionalità del sistema sono organizzate in servizi, ciascuno dei quali viene erogato da un server separato. I client sono utenti di questi servizi e accedono ai server per utilizzarli.
Esempio	La Figura 6 mostra un esempio di biblioteca di film e video/DVD organizzata come sistema client-server.
Quando viene utilizzato	Si utilizza quando i dati di un database condiviso devono essere accessibili da diverse postazioni. Poiché i server possono essere replicati, può essere utilizzato anche quando il carico su un sistema è variabile.
Vantaggi	Il vantaggio principale di questo modello è che i server possono essere distribuiti su una rete. Le funzionalità generali (ad esempio, un servizio di stampa) possono essere disponibili per tutti i client e non devono essere implementate da tutti i servizi.
Svantaggi	Ogni servizio è un <i>single point of failure</i> , quindi è suscettibile di attacchi <i>denial of service</i> o di guasti al server. Le prestazioni possono essere imprevedibili perché dipendono dalla rete e dal sistema. Possono esserci problemi di gestione se i server sono di proprietà di organizzazioni diverse.

Un'architettura client-server per una cineteca

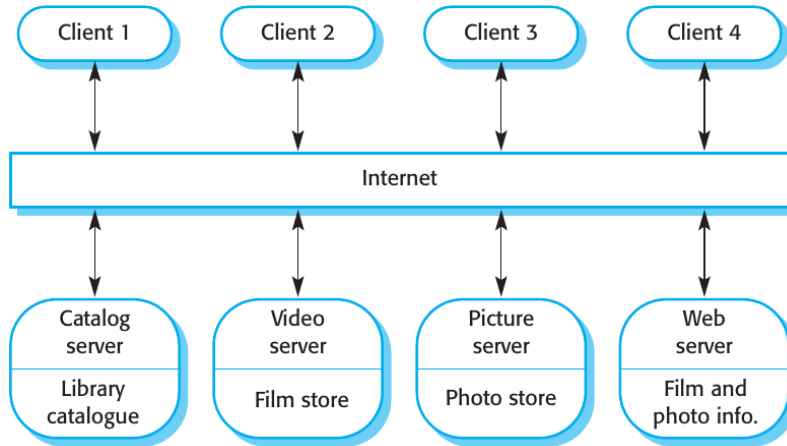


Figura 6

Architettura Pipe and Filter

- Le trasformazioni funzionali elaborano i loro input per produrre output.
- Può essere indicato come modello **pipe and filter** (come nella shell UNIX).
- Varianti di questo approccio sono molto comuni. Quando le trasformazioni sono sequenziali, si tratta di un modello **batch sequential**, ampiamente utilizzato nei sistemi di elaborazione dati.
- Non è adatto ai sistemi interattivi.

Lo schema dei Pipe and Filter

Nome	Pipe and Filter
Descrizione	L'elaborazione dei dati in un sistema è organizzata in modo che ogni componente di elaborazione (filtro) sia discreto ed esegua un tipo di trasformazione dei dati. I dati fluiscono (come in un tubo, pipe) da un componente all'altro per essere elaborati.
Esempio	La Figura 7 mostra un esempio di sistema pipe and filter utilizzato per l'elaborazione delle fatture.
Quando viene utilizzato	Nelle applicazioni di elaborazione dati (batch/transazioni) in cui gli input vengono elaborati in fasi separate per generare output correlati.
Vantaggi	Facile da capire e supporta il riutilizzo delle trasformazioni. Lo stile del flusso di lavoro corrisponde alla struttura di molti processi aziendali. L'evoluzione con l'aggiunta di trasformazioni è semplice. Può essere implementato come sistema sequenziale o concorrente.
Svantaggi	Il formato per il trasferimento dei dati deve essere concordato tra le trasformazioni comunicanti. Ciascuna trasformazione deve analizzare il proprio input e disimparare il proprio output nella forma concordata. Ciò aumenta l'overhead del sistema e può comportare l'impossibilità di riutilizzare trasformazioni funzionali che utilizzano strutture di dati incompatibili.

Un esempio di architettura pipe and filter utilizzata in un sistema di pagamenti

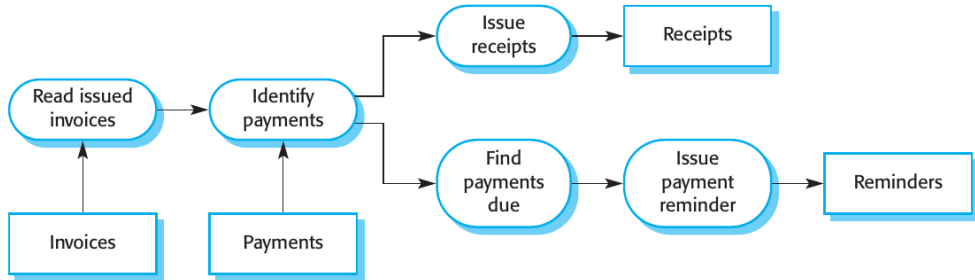


Figura 7